

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Multidisciplinary Project Report

Custom Instruction Set Extension for Int8 Softmax Acceleration on Scalar RISC-V for Edge AI

Master's Degree inComputer Science and Engineering

Author: Luciano Stupnik

Advisor: Tommaso Spagnolo

Co-advisor: Prof. Cristina Silvano

Academic year: 2025-2026

1. Introduction

The softmax function is a fundamental operation in modern deep learning, particularly in Transformer-based architectures. Historically, this function has not been the focus of acceleration efforts, since architectures such as CNNs, MLPs and LSTMs use it only in the final classification layer, where its computational cost is negligible compared to other matrix-intensive operations. In these models, performance has traditionally been dominated by matrix-matrix and matrix-vector multiplications, which are now heavily optimized and accelerated through dedicated MAC units and highly parallel hardware. However, with the growing popularity of the Transformer based architectures for Natural Language Processing (NLP), with applications such as speech recognition, summarization, language understanding, and video captioning [2], accelerating the softmax function has grown in importance, as Transformers rely on softmax within every self-attention block, and as the cost of linear operations continues to decrease due to hardware acceleration, non-linear operations such as softmax are becoming a central performance bottleneck [4].

Moreover, attention-based models have expanded far beyond NLP and are now widely used in Computer Vision (CV) tasks such as object detection, image transformation, and image classification, as well as in more specialized domains including molecule property prediction, symbolic integration, and the solution of differential equations. This broad adoption further amplifies the relevance of optimizing the softmax computation, as the function is repeatedly computed across multiple attention layers in these diverse applications [2].

On top of architectural optimization, research into the compression of Transformers has shown that 8-bit quantized models are able to vastly improve the performance compared to their floating-point counterpart with a minimal loss in accuracy. This finding highlights the robustness of Transformers under reduced numerical precision and opens the door to low-precision hardware implementations of softmax without significant loss of accuracy [3].

Given the new found necesity to accelerate the softmax function, this work introduces an *ITA* [3] and *Softermax* [4] inspired custom instruction to accelerate the softmax computation on RISC-V architectures. The instruction is implemented and evaluated in the gem5 simula-

tor [1], demonstrating significant performance improvements over a baseline RISC-V implementation. To support the new custom instructions an additional Functional Unit (FU) has been integrated in the Execution Stage of the gem5-simulated RISC-V core.

2. Design

This section describes the design of the integer-only softmax implementation, the identification of its computational bottlenecks, and the architecture of the proposed RISC-V custom instruction.

2.1. Integer-Only Softmax Using Shift-Based Exponent Approximation

The softmax function is defined as:

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}.$$

In most cases, this expression is replaced with its numerical stable alternative, defined as:

$$\text{softmax}(x)_i = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^{n} e^{x_j - \max(x)}}.$$

Given that a direct implementation of the function would require floating-point exponentials and division, requiring significant computational power, we decided to follow the principles introduced in ITA [3] to adopt an integer-only implementation. In this formulation the exponentials are replaced by powers of two, differences are bucketized into integer ranges and normalization is performed online.

2.1.1 Input Quantization

Following the ITA approach [3], the inputs to the softmax are quantized using a B-bit fixed-point representation

$$x = \varepsilon x_q,$$

where the scaling factor ε is chosen to maximize the number of non-zero softmax outputs after quantization. ITA shows that the largest meaningful scaling factor is

$$\varepsilon = \frac{B}{2^B \log_2 e}.$$

Under this quantization, the exponential operation can be rewritten in base 2 as

$$e^x = e^{\varepsilon x_q} = 2^{(\log_2 e)\varepsilon x_q} = 2^{\varepsilon' x_q}$$

where

$$\varepsilon' = (\log_2 e)\varepsilon = \frac{B}{2^B}.$$

Using the numerically stable formulation, the following expression of softmax can be derived:

$$\begin{aligned}
\text{softmax}(x)_i &= \frac{2^{\varepsilon'(x_{q,i} - \max(x_q))}}{\sum_j 2^{\varepsilon'(x_{q,j} - \max(x_q))}} \\
&= \frac{2^{\frac{B}{2^B}(x_{q,i} - \max(x_q))}}{\sum_j 2^{\frac{B}{2^B}(x_{q,j} - \max(x_q))}} \\
&= \frac{2^{-\frac{B}{2^B}(\max(x_q) - x_{q,i})}}{\sum_j 2^{-\frac{B}{2^B}(\max(x_q) - x_{q,j})}} \\
&= \frac{2^{-d_i / 2^{B - \log_2 B}}}{\sum_j 2^{-d_j / 2^{B - \log_2 B}}}
\end{aligned} \tag{1}$$

where

$$d_p = \max(x_q) - x_{q,p} \geq 0.$$

2.1.2 Power-of-Two Approximation

For any real-valued input $z = d_p / 2^{B - \log_2 B}$, we approximate the exponential in softmax as:

$$2^{-z} \approx 2^{-k},$$

where k is an integer obtained by rounding the exponent difference d_p, scaled by 2^{K_A}, to the nearest integer. This uniform quantization of exponent differences is referred to as *bucketization*.
The bucket index is computed as shown in algorithm 1:

Algorithm 1 Bucketization of exponent differences

```
function BUCKET(Δ)
    k ← ⌊(Δ + 2^(K_A − 1)) / 2^K_A⌋
    return k
end function
```

where K_A controls the bucket width (i.e., how aggressively exponent differences are grouped). ITA uses a shift-based exponent formulation in

which exponent differences are grouped by a right shift of $(B - \log_2 B)$ bits, effectively suppressing contributions that would quantize to zero. In our bucket-based formulation, this corresponds directly to choosing $K_A = (B - \log_2 B)$. For the quantization with $B = 8$ bits that we are using, we obtain $K_A = 5$. [3]
Finally, the main advantage of this formulation is that 2^{-k} maps directly to a right-shift operation in fixed-point arithmetic:

$$2^{-k} \cdot (1.0)_{\text{Q0.7}} = (1.0)_{\text{Q0.7}} \gg k,$$

allowing the exponential to be computed using only integer shifts, which are extremely efficient in hardware.

2.1.3 Online denominator accumulation

To avoid scanning all the vector to find the maximum first and then scan it again to compute the denominator, we use algorithm 2, which lets do it in one pass. Whenever a new maximum appears, previously accumulated values must be renormalized.

Algorithm 2 Online max and denominator computation (integer domain)

```
function OnlineDenominator(x[1..n])
    m ← x[1]                          ▷ running maximum
    s ← 1.0                           ▷ denominator in Q0.7
    for i = 2..n do
        Δ ← m − x[i]
        k ← Bucket(Δ)
        if x[i] ≤ m then
            s ← s + 2^{−k}
        else
            s ← (s ≫ k) + 1.0
            m ← x[i]
        end if
    end for
    return (m, s)
end function
```

This algorithm mirrors the renormalization strategy used in Softermax [4] where the denominator is shifted whenever a new maximum is discovered to preserve numerical equivalence without storing prior terms.

2.1.4 Integer-only normalization

Once the algorithm 2 has been executed, and given that s is available in fixed-point representation, only a single reciprocal operation is needed:

$$\text{inv} = \frac{1}{s}.$$

Then, given that the numerator terms are computed as 2^{-k_i}, the final probability of each element can be computed as:

Algorithm 3 Final integer-domain normalization

```
function Normalize(x[1..n], m, s)
    inv ← fixed-point reciprocal of s
    for i = 1..n do
        Δ ← m − x[i]
        k ← Bucket(Δ)
        numerator ← 2^{−k}
        y[i] ← numerator × inv
    end for
    return y
end function
```

This formulation reduces softmax to three primitive integer operations: subtraction, shifts, and fixed-point multiplication. These operations are hardware-friendly and naturally suited for custom RISC-V ISA extensions and lightweight functional units.

2.2. Bottleneck Analysis of Integer-Only Softmax

To find the bottleneck of the integer-only softmax implementation described in the last section, we implemented the algorithm in C and compiled it in RISC-V using GCC. After that, we run it on gem5 and analyzed the simulation's output to find the statistics described in Table 1.

Metric	1024 el.	8192 el.	32768 el.
FU Busy (% of cycles)			
IntAlu Busy (%)	68.28%	27.74%	10.07%
IntMult Busy (%)	0.00%	0.00%	0.00%
IntDiv Busy (%)	0.00%	0.00%	0.00%
DenomOP Busy (%)	0.00%	0.00%	0.00%
OpClass Distribution (%)			
IntAlu	69.53%	62.70%	61.37%
MemRead	21.25%	24.84%	25.53%
MemWrite	8.78%	11.76%	12.34%
IntMult	0.42%	0.70%	0.76%
ecall	0.00%	0.00%	0.00%
FloatMemWrite	0.00%	0.00%	0.00%
IntDiv	0.00%	0.00%	0.00%
IntAlu Operation Breakdown			
ADD-like (%)	35.46%	46.88%	49.13%
SHIFT-like (%)	15.30%	16.20%	16.54%
LOGIC (%)	8.72%	5.91%	5.35%
BRANCH (%)	18.91%	11.33%	9.63%
MOVE (%)	11.55%	14.43%	15.12%
OTHER (%)	10.04%	5.24%	4.23%

Table 1: Comparison of gem5 Statistics for Different Vector Sizes

Being that in all the cases, the IntAlu is the only functional unit that is busy, and having the highest percentage of instructions executed, we have decided to focus on optimizing the ADD-like and SHIFT-like operations, which are the most frequent in the IntAlu. That is why out custom instruction proposed in the next section focuses on optimizing the computation used for the online denominator and sum accumulator, described in Algorithm 2, where this operations are mostly used.

3. Custom RISC-V Instruction Specification

This custom instruction extends the RV64GC ISA to accelerate the Softmax operation by performing, in a single instruction, multiple steps of Algorithm 2.

3.1. Register Usage in the Instruction

The custom instruction uses packed integer data contained in the source registers. This design minimizes memory traffic and enables the instruction to update the Softmax accumulator state in less instructions. When the instruction is called, `Rs2` should be loaded with up to four 8-bit input elements of the vector to be processed, while `Rs1` carries the current running state of the Softmax computation (current maximum and the partial denominator accumulated so far) and the number of elements loaded on `Rs2`.

The bit-level layouts of the two input registers are shown in Tables 3 and 2.

31–24	23–16	15–0
count (1–4)	running max (int8)	partial sum (Q0.7, uint16)

Table 2: Bit-level layout of `Rs1`, encoding vector length and the Softmax running state.

31–24	23–16	15–8	7–0
elem4 (int8)	elem3 (int8)	elem2 (int8)	elem1 (int8)

Table 3: Bit-level layout of `Rs2`, containing up to four 8-bit input elements.

3.2. Instruction Opcode

The opcode of the custom Softmax instruction defined, **OnlineDM**, is showed in table 4.

31–25	24–20	19–15	14–12	11–7	6–0
funct7	rs2	rs1	funct3	rd	opcode
`0000000`	`rs2[4:0]`	`rs1[4:0]`	`000`	`rd[4:0]`	`0001011`

Table 4: R-type encoding of OnlineDM.

To execute the instruction, the generic GNU assembler directive `.insn` can be used, without the need for a dedicated mnemonic. The invocation format for an R-type instruction is shown below:

```
.insn r opcode, funct3, funct7, rd,
        rs1, rs2
```

In the case of **OnlineDM**, the specific encoding is:

```
.insn r 0x0b, 0, 0, rd, rs1, rs2
```

3.3. Latency and Functional Unit Design

After an extensive analysis of possible hardware implementations for these operations, the FU in gem5 has been modeled as follows, with a latency of three cycles (the latency used to obtain the results presented in Section 4). In **Stage 1**

the unit unpacks the data from `Rs1`/`Rs2`, masks the invalid elements (if count < 4) and computes the two pairwise maxima (x_0, x_1) and (x_2, x_3) in parallel. In **Stage 2** it computes the local maximum, compares it with the current global maximum m and, if needed, rescales the accumulated sum s by a simple right shift. Finally, in **Stage 3** it computes the four elements contributions and reduces them with a small adder tree, again largely in parallel, before repacking the updated (s', m') into the destination register.

Because each stage only performs light integer work, a three-cycle latency could be possible using the proposed design. Nevertheless, different latencies have been tested in gem5 to see their impact on performance. This was done because knowing how much the performance of the FU affects the overall performance of the instruction might be important to the designer, given that there are many diffent ways in which the FU could be implemented. It is shown in table 6 that the difference between a latency of 2, 3, 4 OR 8 cycles is negligible, given the small percentage of improvement that is reduced in each case, allowing more freedom in the final design of the FU. This happens becuase the custom instruction is a small part of all the integer-only softmax implementation, and even if it helps us accelerate the operation, it is only executed a limited number of times, specifically $\lceil (N_{\text{elements}} - 1)/4 \rceil$ times.

4. Results

The experiments were performed using gem5 v25.0 in system-call (SE) mode. The simulation used a single-core O3 out-of-order RISC-V processor running at 1 GHz, together with private 32 kB L1 instruction and data caches and a unified 256 kB L2 cache. The memory system consisted of a single-channel DDR3-1600 model with a total capacity of 2 GB. Both the baseline and the custom-instruction kernels were cross-compiled with `riscv64-linux-gnu-gcc` at the default optimization level (`-O0`).

The statistics obtained using the custom instruction with a latency of 3 cycles are shown in table 5.

Metric	1024 el.	8192 el.	32768 el.
FU Busy (% of cycles)			
IntAlu Busy (%)	59.05%	28.44%	10.33%
IntMult Busy (%)	0.00%	0.00%	0.00%
IntDiv Busy (%)	0.00%	0.00%	0.00%
DMUpdateOp Busy (%)	0.00%	0.00%	0.00%
OpClass Distribution (%)			
IntAlu	70.01%	61.50%	59.60%
MemRead	21.26%	25.93%	26.97%
MemWrite	8.12%	11.42%	12.16%
IntMult	0.49%	0.92%	1.01%
DMUpdateOp	0.12%	0.23%	0.25%
ecall	0.01%	0.00%	0.00%
FloatMemWrite	0.01%	0.00%	0.00%
IntDiv	0.00%	0.00%	0.00%

Table 5: FU busy and OpClass distribution for different vector sizes using the custom instruction (latency = 3).

It can be noticed that in table 5, we have a new functional unit named **DMUpdateOp**, which is the one that executes the custom instruction. The resutls of the non-custom implementation can be seen in Table 1.

Finally, the total execution time and the improvement obtained on each latency comparing the custom and non-custom implementations can be seen in table 6. Execution time is reported in SimTicks, which is the total number of simulated clock ticks in gem5.

Metric	1024 el.	8192 el.	32768 el.
Non-custom	134.77	732.52	2778.18
Latency = 2			
Custom	105.39	482.45	1778.49
Improvement [%]	21.80	34.14	35.98
Latency = 3			
Custom	105.63	484.74	1786.73
Improvement [%]	21.62	33.83	35.69
Latency = 4			
Custom	105.89	486.69	1794.93
Improvement [%]	21.43	33.56	35.39
Latency = 8			
Custom	106.88	494.69	1827.432
Improvement [%]	20.69	32.47	34.22

Table 6: SimTicks (in 10^6) for custom and non-custom implementations.

The improvement percentage with respect to the baseline is defined as

$$\text{Improvement} = \left(1 - \frac{T_{\text{cust}}}{T_{\text{base}}}\right) \times 100,$$

where T_{cust} and T_{base} denote the number of simulation ticks of the custom and non-custom implementations, respectively.
As already mentioned in subsection 3.3, table 6 allows designers to evaluate the impact of different latencies for the custom instruction, and carefully choose the best trade-off between complexity and performance.

5. Conclusions

The experiments show that adding a custom RISC-V instruction for softmax brings a clear and consistent performance benefit. Across all vector sizes, execution time drops by **20–36%**, with the largest improvements appearing when the vectors are longer.
Another important observation is that the design is **insensitive to latency**. Whether the instruction takes 2, 3, or even 8 cycles, the overall execution time barely changes. This gives flexibility on the hardware implementation side without hurting performance.
Finally, we observe a clear dependence on **vector size**. The improvement increases from about 20.69% for 1 024 elements to 34.22% for 32 768 elements (8-cycle latency). However, it is important to note that the difference in improvement between a vector of 8 192 and 32 768 elements is only of 1.75%, showing a clear nonlinear trend with a reduced improvement rate at larger vector sizes.
Overall, the custom instruction is simple, effective, and scales well with workload size. It provides a practical way to speed up softmax on a scalar RISC-V core and can meaningfully reduce execution time in attention-heavy applications.

References

[1] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39:1–7, 2011.

[2] Wooseok Chang and Ihor Vasyltsov. Efficient softmax approximation for deep neural networks with attention mechanism. *arXiv preprint arXiv:2111.10770*, 2021.

[3] Gianna Paulin, Tim Fischer, Victor J. B. Jung, Angelo Garofalo, Luca Benini, Gamze Islamoglu, and Moritz Scherer. Ita: An energy-efficient attention and softmax accelerator for quantized transformers. *arXiv preprint arXiv:2307.03493*, 2023.

[4] Jacob R. Stevens, Rangharajan Venkatesan, Steve Dai, Brucek Khailany, and Anand Raghunathan. Softermax: Hardware/software co-design of an efficient softmax for transformers. *arXiv preprint arXiv:2103.09301*, 2021.